



The AI Coding Governance Kit

Boris Cherny Framework — Slash Commands for Claude Code & Codex CLI

By Max Dion — CEO, AXKI
AI Automation Consultancy — Montreal, Canada

axki.ca | max@axki.ca

Version 1.0 — April 2026

Table of Contents

1.	Why AI Coding Governance Matters	3
2.	The Boris Cherny Framework	4
3.	Framework 1: Quick Checklist	5
4.	Framework 2: Plan Validator	6
5.	Framework 3: Adversarial Watchdog	8
6.	Role-Agnostic Architecture	9
7.	Installation Guide	10
8.	Real-World Validation (Screenshot)	11
9.	Next Steps — Book a Discovery Call	12

1. Why AI Coding Governance Matters

AI coding assistants are powerful — but without governance, they become liability generators. Left unchecked, AI agents will hallucinate file references, claim work is done when it isn't, bundle unrelated changes into monolithic commits, skip testing, and push untested code to production.

The result? Technical debt that compounds faster than any human team could create. Phantom features that exist only in the AI's output. Tests that pass because they test nothing. And a codebase that looks productive on the surface but crumbles under scrutiny.

Boris Cherny's framework solves this by imposing a strict phase-gate protocol on AI coding sessions. Every phase requires explicit human approval (GO) before proceeding. Every commit must be atomic. Every claim must be verifiable. And a dedicated adversarial reviewer catches what the builder misses.

Common AI Coding Failures (Without Governance)

Failure Mode	Impact	BC Framework Solution
Hallucinated file references	Broken builds, phantom features	Watchdog Challenge D
Skipped planning phase	Scope creep, wasted cycles	HITL Gates 1-2
Bundled commits	Impossible rollbacks	Atomic commit rule
Tests that test air	False confidence	Watchdog Challenge C
No memory updates	Context loss between sessions	Memory discipline checks
Self-validated work	Confirmation bias	Adversarial reviewer role

2. The Boris Cherny Framework

The Boris Cherny framework enforces a strict sequential loop for AI-assisted development. Each phase has a clear entry gate, defined deliverables, and requires explicit human approval before the next phase can begin. No shortcuts. No skipping.

The Four Phases

Phase	Purpose	Gate	Deliverable
EXPLORE	Understand the problem space	GATE 1 → GO	Problem definition, constraints
PLAN	Design the solution	GATE 2 → GO	Approved implementation plan
IMPLEMENT	Write code in lanes	GATE 3 → GO	Tested code, all green
COMMIT	Atomic commits, push	GATE 4 → GO Prod	Clean git history, deployed

Core Principles

- **Sequential phases** — Explore → Plan → Implement → Commit. Never skip.
- **HITL gates** — Human-in-the-loop approval required between every phase.
- **Atomic commits** — One concern per commit. No bundling unrelated changes.
- **Real worktrees** — Use `git worktree add` inside `buildup/lanes/`. Not folders.
- **Stop-hooks** — Define test commands. Loop until all green. No premature success.
- **Memory discipline** — Update governance files (CLAUDE.md / AGENTS.md) every session.
- **Adversarial review** — A dedicated reviewer audits the executor before any merge.

3. Framework 1: Quick Checklist

Command: `/axki:checklist`

A rapid governance audit to run before any commit or phase transition. The reviewer scores each item against the executor's work using: ✓ PASS | ✗ FAIL | ■ PARTIAL.

PHASE & LOOP

- ☐ Phase identified? (Explore/Plan/Implement/Commit)
- ☐ Loop order respected? No skipping.
- ☐ Plan Mode active before any writes?
- ☐ Explicit GO received before this phase started?

LANES & WORKTREES

- ☐ Real Git worktrees – not folders?
- ☐ Worktrees inside buildup/lanes/ – not siblings?
- ☐ Each lane has one concern only?
- ☐ No cross-lane contamination?
- ☐ Worktrees removed after each lane merges?

GATES

- ☐ GATE 1 cleared? (Explore → GO)
- ☐ GATE 2 cleared? (Plan → GO)
- ☐ GATE 3 pending? (Implement → GO before Commit)
- ☐ GATE 4 active? (No prod without GO Prod)

DONE CRITERIA

- ☐ test_cmd defined?
- ☐ done_when defined per task?
- ☐ Stop-hook loops until all green?
- ☐ Failure protocol: fix → new commit → rerun?

COMMITTS

- ☐ Atomic? One concern per commit?
- ☐ No bundled commits?
- ☐ No half-open migrations?

MEMORY & GOVERNANCE

- ☐ Memory file updated this session?
- ☐ ÉTAT COURANT reflects actual state?
- ☐ Mistakes logged in ERREURS CONNUES?
- ☐ Session log written to /logs/?
- ☐ Merged branches deleted?
- ☐ git push origin master executed?

ANTI-PATTERNS

- ☐ No monolithic scripts?
- ☐ No phantom file references?
- ☐ No "I wrote X" without proof of write?
- ☐ No tests that test air (mock always true)?
- ☐ No LLM output assumed correct?
- ☐ No over-engineering beyond current scope?

VERDICT: GO / NEEDS REVISION / BLOCK

4. Framework 2: Plan Validator

Command: /axki:validate

Validate a proposed plan against the full Boris Cherny protocol before the executor begins implementation. The reviewer reads the plan and scores each governance criterion. This is the most comprehensive check — run it before any GATE 2 approval.

LOOP & PHASE

- ☐ Current phase clearly identified?
- ☐ Order respected? No Implement without approved Plan.
- ☐ Plan Mode active? No file writes before GO.
- ☐ One clear GO gate before the next phase?

LANES & WORKTREES

- ☐ Real Git worktrees — explicit git worktree add commands?
- ☐ All worktrees inside buildup/lanes/?
- ☐ Each lane named and assigned before Implement?
- ☐ Each lane scoped to one concern?
- ☐ Worktrees created one at a time?
- ☐ Removal after each merge planned?

HITL GATES

- ☐ GATE 1 present? (Explore complete → wait for GO)
- ☐ GATE 2 present? (Plan approved → wait for GO)
- ☐ GATE 3 defined? (Implement done → GO before Commit)
- ☐ GATE 4 respected? (No push to prod without GO Prod)

STOP-HOOKS / DONE CRITERIA

- ☐ Done-when defined before Implement starts?
- ☐ Stop-hook command specified per task?
- ☐ Will agent loop until all green?
- ☐ Failure protocol: fix → new commit → rerun?
- ☐ Monotonic test count enforced?

COMMITTS

- ☐ Atomic? One concern per commit?
- ☐ No bundled commits planned?
- ☐ No half-migrations left open?

ADVERSARIAL REVIEW

- ☐ Reviewer agent defined in workflow?
- ☐ Review before each lane merge?
- ☐ Final verification after session planned?

MEMORY

- ☐ Memory file update planned at session end?
- ☐ ERREURS CONNUES update planned?
- ☐ Session log to /logs/ planned?
- ☐ git push origin master in governance?
- ☐ Merged branches deletion planned?

ANTI-PATTERNS CHECK

- ☐ No monolithic scripts?
- ☐ No skipped Plan Mode?
- ☐ No prod push without GO Prod?
- ☐ No LLM output assumed correct?
- ☐ No over-engineering beyond current scope?
- ☐ No phantom file references?

SECURITY (if OAuth or external API)

- ☐ State validation planned (Redis TTL)?
- ☐ Public paths exempted for callbacks?
- ☐ CSRF exemption for webhooks/callbacks?
- ☐ New users assigned to correct tenant only?
- ☐ JWT parity with existing auth flow?

Fails found: ____ | Partial found: ____

VERDICT: GO / NEEDS REVISION / BLOCK

5. Framework 3: Adversarial Watchdog

Command: `/axki:watchdog`

The most aggressive audit tool. The reviewer assumes the executor is trying to look productive and systematically proves or disproves that claim. This is not a code review — it's a lie-detection protocol for AI agents.

Reviewer mindset: Senior adversarial watchdog. Your job: catch lies, gaps, hallucinations, and protocol violations. You are NOT a builder. You are NOT here to suggest code. Assume the executor is trying to look productive. Prove or disprove that.

CHALLENGE A — FILE & MEMORY DISCIPLINE

- ☐ Did executor actually write to memory file?
- ☐ Did executor update ÉTAT COURANT?
- ☐ Did executor update ERREURS CONNUES?
- ☐ Did executor write output to correct files with confirmation read?
- ☐ Did executor push to origin/master?
- ☐ Did executor write session log?

CHALLENGE B — TASK PLAN COMPLIANCE

- ☐ List every task from the approved plan.
- ☐ For each task: DONE / PARTIAL / NOT DONE / CLAIMED BUT UNVERIFIED.
- ☐ Flag any task marked done without visible proof.
- ☐ Flag scope creep: work not in approved plan.
- ☐ Flag silent drops: plan items with no explanation.

CHALLENGE C — TEST VALIDITY

- ☐ Did the tests actually run? Show terminal output.
- ☐ Are test assertions meaningful?
- ☐ Did any test fail? Zero failures on complex implementation is suspicious.
- ☐ Is test count monotonically increasing?

CHALLENGE D — HALLUCINATION DETECTION

- ☐ Did executor reference a file never shown?
- ☐ Did executor claim a feature works without real execution trace?
- ☐ Did executor say "this is implemented" for any known gap?
- ☐ Did executor describe future work as current work?

CHALLENGE E — BORIS CHERNY PROTOCOL

- ☐ Did executor exit Plan Mode before GO?
- ☐ Were commits atomic?
- ☐ Did any lane work happen outside assigned worktree?
- ☐ Were worktrees inside buildup/lanes/?
- ☐ Were merged worktrees removed?
- ☐ Were merged branches deleted?

[] Was GATE 4 respected?

FINAL WATCHDOG REPORT

MEMORY & FILE DISCIPLINE: [VERIFIED / PARTIAL / FAILED]

TASK PLAN COMPLIANCE: [VERIFIED / PARTIAL / FAILED]

TEST VALIDITY: [REAL / THEATER / UNVERIFIED]

HALLUCINATIONS DETECTED: [YES – list / NONE FOUND]

BC PROTOCOL VIOLATIONS: [list each / NONE FOUND]

FINAL VERDICT: GO / NEEDS REVISION / BLOCK

6. Role-Agnostic Architecture

The AI landscape shifts constantly. GPT-5.5, Sonnet 4.8, Opus 4.7 — new models drop every quarter. Hardcoding tool names into governance frameworks creates brittle systems that break with every update.

The AXKI Boris Cherny plugin solves this with a **role-agnostic architecture**. At session start, the user declares who is the executor (builder) and who is the reviewer (validator). The frameworks reference roles, not tools. Swap freely without reinstalling.

The `/axki:init` Command

```
/axki:init executor=codex reviewer=claude
```

```
/axki:init executor=claude reviewer=codex
```

Role	Responsibilities	Commands Used
EXECUTOR (Builder)	Write code, create commits, run tests, manage worktrees, update memory files	Receives verdicts from reviewer
REVIEWER (Validator)	Validate plans, run checklists, deploy watchdog, issue GO/BLOCK verdicts, catch hallucinations	<code>/axki:checklist</code> <code>/axki:validate</code> <code>/axki:watchdog</code>

This means when a new model drops — whether it's Claude 4, GPT-6, or Gemini Ultra — you simply change the init command. The governance framework stays intact. No file edits. No reinstall. No migration.

7. Installation Guide

For Claude Code (Skills Format)

Claude Code uses a `.claude/skills/` directory in your project root. Each skill is a Markdown file that defines a slash command.

```
# Clone the repo git clone
https://github.com/AXKI-Consulting/axki-boris-cherny-governance.git # Copy skills to
your project cp -r axki-boris-cherny-governance/.claude/ /path/to/your/project/
```

Available commands after install:

- `/skill:axki-init` – Initialize executor/reviewer roles
- `/skill:axki-checklist` – Quick governance checklist
- `/skill:axki-validate` – Plan validator
- `/skill:axki-watchdog` – Adversarial watchdog

For Codex CLI (AGENTS.md Format)

Codex CLI uses an `AGENTS.md` file in your project root with command files in a `.codex/commands/` directory.

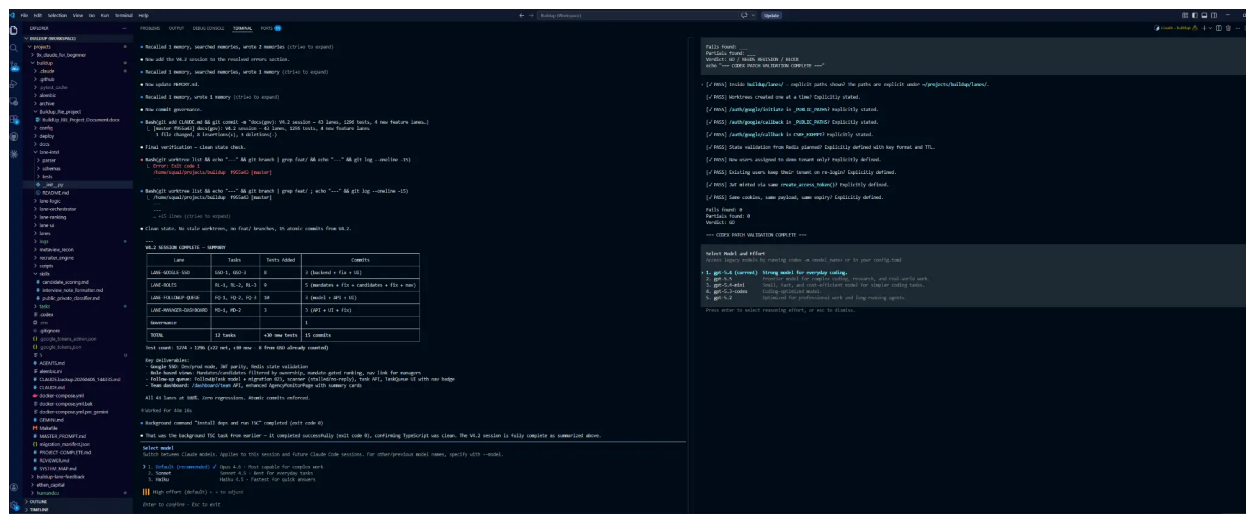
```
# Clone the repo git clone
https://github.com/AXKI-Consulting/axki-boris-cherny-governance.git # Copy Codex
config to your project cp axki-boris-cherny-governance/codex/AGENTS.md
/path/to/your/project/ cp -r axki-boris-cherny-governance/codex/commands/
/path/to/your/project/.codex/commands/
```

Available commands after install:

- `/codex:axki-init` – Initialize executor/reviewer roles
- `/codex:axki-checklist` – Quick governance checklist
- `/codex:axki-validate` – Plan validator
- `/codex:axki-watchdog` – Adversarial watchdog

8. Real-World Validation

Below is a real screenshot from a production session where Codex CLI acts as the executor and Claude Code acts as the adversarial reviewer. The watchdog validates every claim, checks test output, and issues a final verdict.



Codex executor validated by Claude Code reviewer — adversarial watchdog catching protocol violations in real-time.

Session summary shows 12 tasks, 30+ tests, 15 atomic commits.

This is what governance looks like in practice. The reviewer doesn't trust the executor's claims — it verifies them independently. Every PASS has evidence. Every FAIL has an explanation. The final verdict is earned, not assumed.

9. Ready to Implement This?

These frameworks are free and open-source. But implementing AI governance across a real engineering team — with custom lanes, project-specific stop-hooks, and multi-agent workflows — requires hands-on expertise.

AXKI helps engineering teams deploy structured AI coding governance so they ship faster without losing control. We've battle-tested these frameworks across production codebases and can adapt them to your specific stack and workflow.

Book a Free Discovery Call with Max
Let's discuss how to implement Boris Cherny
governance
in your AI-assisted development workflow.

axki.ca | max@axki.ca

AXKI — AI Automation Consultancy

Montreal, Canada

